



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Carthage

Institut Supérieur des Technologies de
l'Information et de la Communication



RAPPORT DE STAGE D'ÉTÉ

Présenté en vue de l'obtention du

DIPLÔME D'INGÉNIEUR EN [SPÉCIALITÉ À COMPLÉTER]

Parcours : [parcours à compléter]

Par

[PRÉNOM NOM]

Plateforme d'ingénierie de détection CTI assistée par intelligence artificielle

Réalisé au sein de Forvis Mazars Group

Emplacement réservé au logo
Forvis Mazars Group

Encadrant professionnel : [Prénom NOM, fonction]

Encadrant académique : [Prénom NOM, grade]

Année universitaire : 2024–2025

Durée du stage : un mois

Fiche de validation du stage

Étudiant	[Prénom NOM]
Organisme d'accueil	Forvis Mazars Group
Durée	Un mois
Sujet	Plateforme d'ingénierie de détection CTI assistée par intelligence artificielle
Encadrant professionnel	[Nom et fonction à compléter]
Encadrant académique	[Nom et grade à compléter]

Visa de l'organisme d'accueil

Visa de l'encadrant académique

Signature et cachet

Signature

Remerciements

Je tiens à remercier l'ensemble des personnes qui ont contribué au bon déroulement de ce stage réalisé au sein de Forvis Mazars Group.

J'adresse en particulier mes remerciements à mon encadrant professionnel pour son accompagnement, ses orientations et la confiance qui m'a été accordée pendant cette période. Les échanges réguliers ont facilité la compréhension des besoins liés à la Cyber Threat Intelligence et à l'ingénierie de détection.

Je remercie également mon encadrant académique ainsi que les enseignants de l'Institut Supérieur des Technologies de l'Information et de la Communication pour les connaissances acquises au cours de ma formation et pour leurs conseils méthodologiques.

Enfin, je remercie les membres des équipes avec lesquelles j'ai pu échanger. Leurs retours sur les contraintes opérationnelles d'un SOC, la qualité des règles de détection et la nécessité de conserver une validation humaine ont directement enrichi ce travail.

Table des matières

Remerciements	i
Introduction Générale	1
1 Contexte du stage et analyse du besoin	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.3 Contexte métier	4
1.3.1 Du renseignement à la détection	4
1.3.2 Problème identifié	4
1.4 Parties prenantes et rôles	4
1.5 Objectifs fonctionnels	5
1.6 Objectifs techniques et contraintes	5
1.7 Démarche adoptée	6
1.8 Conclusion	6
2 Conception et architecture de la plateforme	7
2.1 Introduction	7
2.2 Concepts techniques mobilisés	7
2.2.1 CTI, MISP et comportements détectables	7
2.2.2 MITRE ATT&CK	8
2.2.3 Sigma et pySigma	8
2.2.4 LangGraph et traitement asynchrone	8
2.3 Architecture générale	8
2.3.1 Frontend	9
2.3.2 Backend	9

2.3.3	MISP, Celery et planification	10
2.4	Architecture des données	10
2.5	Workflow de raisonnement et de décision	11
2.5.1	Étapes du graphe	11
2.5.2	Révision bornée et mémoire de session	12
2.6	Contrôles, confiance et approbation humaine	12
2.6.1	Watchers	12
2.6.2	Score de confiance	12
2.6.3	Score de fiabilité déterministe	13
2.6.4	Principe de contrôle humain	13
2.7	Déploiement conteneurisé	13
2.8	Conclusion	14
3	Réalisation, validation et bilan	15
3.1	Introduction	15
3.2	Mise en œuvre du backend	15
3.2.1	API et sécurité applicative	15
3.2.2	Ingestion MISP et idempotence	15
3.2.3	Services déterministes	16
3.3	Mise en œuvre du moteur d'intelligence artificielle	16
3.4	Mise en œuvre du frontend	17
3.5	Chaîne DevSecOps et CI/CD	17
3.6	Stratégie de test	18
3.6.1	Tests unitaires du backend	18
3.6.2	Tests d'intégration	19
3.6.3	Tests de bout en bout	19
3.7	Difficultés rencontrées et solutions	19
3.8	Résultats obtenus	20
3.9	Limites	20
3.10	Perspectives d'amélioration	21
3.11	Conclusion	21

Conclusion Générale	22
Bibliographie	24
Annexe 1, Référentiel technique de la plateforme	25

Table des figures

2.1	Architecture générale de la plateforme	9
2.2	Workflow de raisonnement de l'intelligence artificielle	11
3.1	Interface de la file de revue analyste	17
3.2	Pipeline DevSecOps de la plateforme	17

Liste des tableaux

2.1	Principaux domaines du modèle de données	10
3.1	Workflows d'intégration et de déploiement continus	18

Introduction Générale

Les centres opérationnels de sécurité reçoivent quotidiennement des informations décrivant des campagnes malveillantes, des infrastructures adverses, des fichiers suspects ou des modes opératoires. Ces données de renseignement sur les menaces, désignées par le terme *Cyber Threat Intelligence* (CTI), ne deviennent toutefois utiles à la surveillance que lorsqu'elles sont transformées en mécanismes de détection adaptés aux journaux réellement disponibles. Cette transformation demande habituellement plusieurs opérations : normalisation des événements, extraction des comportements, association aux techniques MITRE ATT&CK, vérification de la télémétrie, recherche de doublons, rédaction de règles Sigma, compilation, validation et publication.

Lorsque ces opérations sont réalisées manuellement, leur enchaînement peut être long et difficile à reproduire. La qualité dépend en outre de la disponibilité de l'analyste et de sa connaissance simultanée du renseignement, des sources de journaux et des conventions de détection. L'intelligence artificielle générative peut accélérer certaines tâches de synthèse et de rédaction, mais ses résultats ne doivent pas être considérés comme fiables sans contrôle. Une règle plausible sur le plan linguistique peut contenir un champ inexistant, une association ATT&CK incorrecte, une logique trop large ou une information absente de l'événement source.

Le stage d'un mois effectué au sein de Forvis Mazars Group s'inscrit dans ce contexte. Il porte sur une plateforme locale d'ingénierie de détection CTI assistée par intelligence artificielle. La solution reçoit de véritables événements MISP, les normalise, orchestre leur traitement dans un graphe LangGraph, produit des propositions Sigma et les soumet à une série de contrôles déterministes. Elle fournit ensuite à l'analyste une file de revue permettant d'approuver, de rejeter ou de demander une correction. Le principe directeur du projet est le suivant : l'intelligence artificielle peut proposer, réparer et recommander, mais elle ne peut ni approuver ni déployer une détection. La décision finale demeure humaine.

L'objectif de ce rapport est de présenter le besoin, les choix de conception, la réalisation et les résultats obtenus. Le premier chapitre expose le cadre du stage, la problématique et les objectifs fonctionnels et techniques. Le deuxième chapitre décrit les notions nécessaires, l'architecture de la plateforme, le modèle de données et le workflow de rai-

sonnement. Le troisième chapitre détaille la mise en œuvre, les mécanismes de validation, l'intégration DevSecOps, les tests, les difficultés rencontrées ainsi que les limites de la solution. Une conclusion générale synthétise les apports du stage et propose des perspectives d'amélioration.

Chapitre 1

Contexte du stage et analyse du besoin

1.1 Introduction

Ce chapitre présente le cadre dans lequel le projet a été réalisé. Il précise le problème rencontré par les équipes SOC et d'ingénierie de détection, identifie les acteurs du système et traduit le besoin en objectifs fonctionnels et techniques. Cette analyse sert de base aux choix d'architecture présentés dans le chapitre suivant.

1.2 Présentation de l'organisme d'accueil

Le stage a été réalisé au sein de Forvis Mazars Group. Dans le cadre de cette mission, l'environnement étudié concerne les activités de cybersécurité, et plus particulièrement le traitement du renseignement sur les menaces et la production de règles de détection. Le projet se place à l'interface entre les analystes CTI, qui collectent et qualifient l'information sur les adversaires, et les analystes SOC, qui exploitent la télémétrie des systèmes pour identifier des activités suspectes.

La durée limitée du stage, fixée à un mois, a conduit à privilégier une plateforme locale reproductible, démontrant l'ensemble du cycle de traitement sans prétendre remplacer une infrastructure SOC de production. Cette contrainte a orienté la solution vers Docker Compose, des services clairement séparés et des scénarios de test déterministes.

1.3 Contexte métier

1.3.1 Du renseignement à la détection

Une plateforme MISP centralise des événements CTI contenant des attributs tels que des adresses IP, des noms de domaine, des empreintes de fichiers, des commandes ou des descriptions d'activité. Un indicateur de compromission peut être directement recherché, mais sa durée de vie est parfois courte. Une détection fondée sur un comportement, par exemple une technique d'exécution ou de persistance, peut offrir une couverture plus durable. Il faut donc interpréter les éléments de l'événement afin d'identifier des comportements observables et de les traduire en logique de détection.

Cette traduction nécessite plusieurs compétences. L'analyste doit vérifier que le comportement est réellement soutenu par les éléments sources, choisir une technique MITRE ATT&CK pertinente, déterminer les journaux et les champs nécessaires, vérifier la couverture existante, rédiger la règle Sigma puis tester sa compilation vers le langage de la plateforme cible. Chaque étape peut conduire à l'arrêt du traitement : preuves insuffisantes, absence de télémétrie, détection déjà couverte ou règle invalide.

1.3.2 Problème identifié

Le processus manuel comporte de nombreuses ruptures entre outils et plusieurs décisions répétitives. Une information doit être copiée ou reformulée, comparée au catalogue existant, confrontée à l'inventaire de télémétrie puis transformée en règle. L'absence d'une trace structurée rend également difficile l'explication a posteriori d'une décision ou d'une correction.

Le problème traité peut ainsi être formulé de la manière suivante : comment automatiser une part importante de la transformation d'un événement MISP en proposition de détection, tout en garantissant la traçabilité, les contrôles déterministes et l'approbation obligatoire d'un analyste ?

1.4 Parties prenantes et rôles

La plateforme distingue plusieurs responsabilités :

- le système MISP fournit les événements de renseignement ;
- le worker d'automatisation consomme et traite les événements en arrière-plan ;
- le fournisseur d'intelligence artificielle extrait des comportements et propose ou répare des règles ;

- les services déterministes vérifient les preuves, ATT&CK, la télémétrie, les doublons et la validité Sigma ;
- l’analyste consulte les résultats et prend la décision d’approbation, de rejet ou de demande de modification ;
- l’administrateur configure notamment les sources de télémétrie, les paramètres et la planification MISP.

Cette séparation empêche le modèle d’intelligence artificielle de devenir l’autorité de décision. Elle permet aussi d’attribuer clairement chaque résultat à son origine : proposition probabiliste, règle métier déterministe ou action humaine.

1.5 Objectifs fonctionnels

L’objectif principal est de couvrir le cycle de vie complet d’une proposition de détection. La plateforme doit :

1. interroger l’API MISP et normaliser les événements CTI ;
2. créer un workflow indépendant et traçable pour chaque événement ;
3. extraire un ou plusieurs comportements détectables et vérifier leur ancrage dans les preuves ;
4. proposer puis valider les associations MITRE ATT&CK ;
5. comparer le besoin avec le catalogue de détections existant ;
6. vérifier la disponibilité des sources et champs de télémétrie ;
7. décider de poursuivre, d’arrêter ou de générer un candidat ;
8. produire une règle Sigma, la valider et la compiler avec pySigma ;
9. rechercher les doublons exacts et les règles proches ;
10. corriger un candidat défectueux dans une boucle bornée ;
11. conserver les sessions, révisions, contrôles, scores et décisions ;
12. présenter les propositions dans une file de revue ;
13. publier dans le catalogue uniquement après approbation humaine.

1.6 Objectifs techniques et contraintes

Le système doit proposer une API REST authentifiée, une interface destinée au SOC, une base relationnelle comme source de vérité et un traitement asynchrone. L’orchestration doit être assurée par un seul moteur de workflow afin d’éviter des décisions dispersées dans plusieurs composants. La solution doit également pouvoir fonctionner sans appel

à un modèle externe grâce à un mode de données prédéfinies, indispensable pour les démonstrations et l'intégration continue.

Les contraintes principales sont la reproductibilité locale, l'auditabilité et la sécurité. Le déploiement retenu doit réunir la plateforme et une instance MISP réelle tout en séparant leurs réseaux. Les sorties de l'intelligence artificielle doivent être structurées et validables. Les révisions doivent rester immuables afin de préserver l'historique. Enfin, aucune création d'artifact de déploiement ou d'entrée de catalogue ne doit intervenir avant l'action explicite de l'analyste.

1.7 Démarche adoptée

La réalisation suit une progression allant des fondations vers le workflow complet. Les premières activités concernent le modèle de données, l'API, l'authentification et l'environnement conteneurisé. Les services MISP et les contrôles déterministes sont ensuite intégrés. Le graphe de traitement relie ces services aux appels d'intelligence artificielle. Enfin, l'interface de revue, les tests et les pipelines DevSecOps permettent de vérifier le cycle complet.

Cette démarche réduit le risque de construire un workflow reposant uniquement sur des réponses génératives. Les décisions critiques disposent d'abord d'une représentation persistante et de règles déterministes, auxquelles l'intelligence artificielle vient apporter une capacité de proposition.

1.8 Conclusion

L'analyse met en évidence un besoin d'automatisation encadrée plutôt que d'autonomie complète. La plateforme doit accélérer la préparation des détections tout en maintenant une séparation nette entre génération, contrôle et décision. Le chapitre suivant décrit l'architecture retenue pour respecter cette exigence.

Chapitre 2

Conception et architecture de la plateforme

2.1 Introduction

La conception repose sur une architecture distribuée locale associant une application web, des traitements asynchrones, une base de données et une instance MISP. Avant de détailler les composants, il convient de définir les principaux concepts employés dans le workflow.

2.2 Concepts techniques mobilisés

2.2.1 CTI, MISP et comportements détectables

La CTI regroupe les connaissances utiles à la compréhension d'une menace : contexte, indicateurs, outils, cibles et modes opératoires. MISP est utilisé comme source d'événements structurés. Lors de l'ingestion, les données brutes sont conservées mais également transformées en un format interne homogène. Cette normalisation évite que les étapes suivantes dépendent directement de toutes les variantes du format d'origine.

Le système ne se limite pas aux indicateurs isolés. Il cherche à extraire un comportement, c'est-à-dire une action adverse pouvant être observée au moyen d'une télémétrie. Chaque comportement conserve des références vers les attributs qui le justifient. Cette liaison est essentielle pour détecter les affirmations non ancrées dans l'événement source.

2.2.2 MITRE ATT&CK

MITRE ATT&CK fournit une base de connaissances organisée en tactiques et techniques décrivant les actions des adversaires. Une association ATT&CK facilite la classification, la mesure de couverture et la communication entre analystes. Dans la plateforme, l'intelligence artificielle peut proposer une technique, mais la proposition est vérifiée contre une table locale de techniques de référence avant d'être acceptée.

2.2.3 Sigma et pySigma

Sigma est un format générique de règles de détection. Une règle décrit notamment son titre, son statut, ses références, la source de journaux, la logique de sélection, la condition et son niveau. Ce format permet de séparer l'intention de détection du langage propre à un SIEM. pySigma analyse la règle, vérifie sa structure et la compile. Le projet utilise le backend Splunk afin de produire une requête SPL à titre de validation technique.

2.2.4 LangGraph et traitement asynchrone

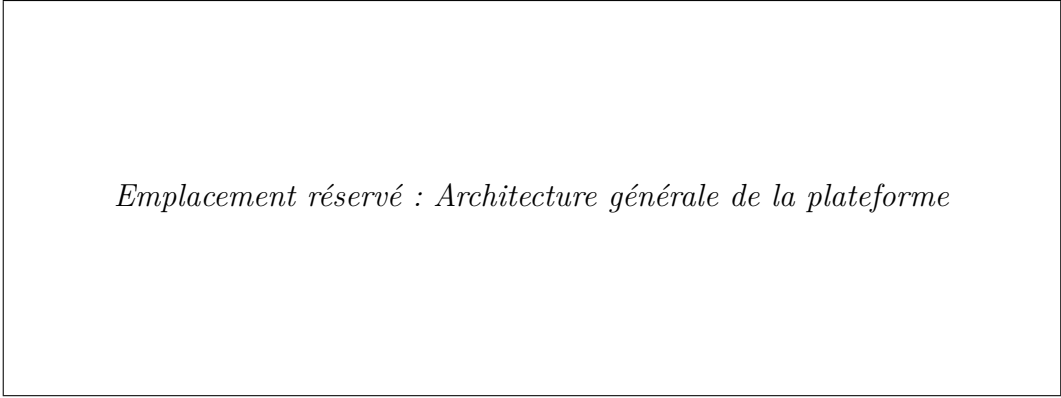
LangGraph permet de représenter un workflow sous forme d'états, de nœuds et de transitions conditionnelles. Contrairement à une suite d'appels linéaires, le graphe peut s'arrêter, boucler vers une correction, traiter plusieurs comportements ou reprendre après une action humaine. Il constitue l'unique moteur de routage du projet.

Celery exécute ce graphe en arrière-plan. Redis sert de courtier de messages et de stockage des résultats Celery. Celery Beat déclenche périodiquement l'interrogation de MISP. Ce découplage évite que les requêtes HTTP restent ouvertes pendant les traitements longs.

2.3 Architecture générale

L'architecture comporte six ensembles principaux : le navigateur de l'analyste, le reverse proxy Nginx, le frontend Next.js, le backend FastAPI, les services asynchrones et les systèmes de données. PostgreSQL conserve l'état métier, tandis que MISP fournit les événements CTI. Redis assure la communication avec les workers.

Le frontend consomme les routes sous le préfixe `/api/v1`. Nginx expose l'ensemble sur le port local 8080. Le backend, le worker et le planificateur partagent l'accès à PostgreSQL et Redis. Ils rejoignent également le réseau interne MISP, contrairement au frontend. Cette topologie limite l'exposition directe de la plateforme de renseignement.



Emplacement réservé : Architecture générale de la plateforme

FIGURE 2.1 – Architecture générale de la plateforme

2.3.1 Frontend

L'interface est développée avec Next.js 14, React 18, TypeScript et TailwindCSS. Elle stocke le jeton JWT dans le stockage local du navigateur et redirige l'utilisateur non authentifié vers la page de connexion. Un client commun centralise les opérations GET, POST, PATCH et le téléchargement d'artifacts.

Les pages couvrent les principales activités : tableau de bord, boîte de réception MISP, événements CTI, sessions d'intelligence artificielle, visualisation du workflow, exécutions du graphe, propositions, file de revue, catalogue de détections, couverture ATT&CK, télémétrie, santé du système, exécutions d'automatisation et paramètres. Des composants réutilisables présentent les métriques, badges d'état, tableaux, chronologies et blocs de code.

2.3.2 Backend

Le backend FastAPI sépare les routes, les schémas Pydantic, les modèles SQLAlchemy, les services et le graphe. L'authentification utilise des jetons JWT avec contrôle des rôles. Les routes exposent les fonctions MISP, CTI, graphe, intelligence artificielle, revue, catalogue, télémétrie, ATT&CK, paramètres et santé.

Les services déterministes constituent l'autorité pour la validation ATT&CK, l'analyse de couverture, la visibilité de télémétrie, la politique de routage, la validation Sigma, la détection de doublons, la confiance et la création d'artifacts. Le fournisseur d'intelligence artificielle est isolé derrière une abstraction offrant un mode prédéfini et un mode connecté à DeepSeek.

2.3.3 MISP, Celery et planification

L'intégration MISP s'appuie sur des images Docker officielles : cœur MISP, modules MISP, MariaDB, Valkey et serveur SMTP. Les responsabilités sont réparties entre cinq services :

- `MispApiService` pour la communication avec l'API ;
- `MispIngestionService` pour la normalisation et l'ingestion ;
- `MispPollingService` pour l'interrogation périodique ;
- `MispConnectivityService` pour le diagnostic ;
- `MispIngestionScheduleService` pour la configuration du planning.

Deux tâches Celery sont exposées. `run_graph` démarre ou reprend un workflow Lang-Graph. `poll_misp` recherche périodiquement de nouveaux événements. L'état de consommation enregistre le déclencheur, la stratégie, les identifiants résolus, les dates et les causes d'échec ou d'omission.

2.4 Architecture des données

PostgreSQL 16 représente la source de vérité. Les tables peuvent être regroupées en six domaines, présentés dans le tableau 2.1.

TABLE 2.1 – Principaux domaines du modèle de données

Domaine	Tables principales
Identité et configuration	<code>users</code> , <code>settings</code> , <code>telemetry_sources</code>
CTI et orchestration	<code>cti_events</code> , <code>workflows</code> , <code>graph_runs</code> , <code>graph_node_runs</code> , <code>cti_consumption_records</code>
Raisonnement IA	<code>ai_interactions</code> , <code>ai_reasoning_sessions</code> , <code>ai_reasoning_revisions</code> , <code>ai_watcher_results</code> , <code>ai_confidence_events</code>
Analyse	<code>behaviors</code> , <code>attack_techniques</code> , <code>attack_mappings</code> , <code>coverage_results</code> , <code>visibility_results</code> , <code>policy_decisions</code>
Revue	<code>proposals</code> , <code>proposal_revisions</code> , <code>validation_results</code> , <code>review_actions</code>
Publication	<code>deployment_artifacts</code> , <code>detection_catalog</code> , <code>detection_attack_mappings</code>

Des contraintes d'unicité empêchent notamment la duplication d'un événement MISP, la création de plusieurs workflows pour le même événement ou la publication multiple du même artifact. Les révisions de propositions sont immuables et numérotées. Les exécutions du graphe et de chaque nœud conservent les entrées, sorties, durées et erreurs, ce qui fournit une piste d'audit détaillée.

2.5 Workflow de raisonnement et de décision

2.5.1 Étapes du graphe

Le graphe débute par `consume_cti`, qui charge l'événement normalisé. Le nœud `extract_behaviors` interroge ensuite le fournisseur d'intelligence artificielle et persiste les comportements. Des contrôles vérifient leur schéma, leurs preuves, la présence d'une proposition ATT&CK, leur ancrage dans les observables et le respect du format structuré.

Pour chaque comportement, `verify_attack_mapping` confirme les techniques proposées. `coverage_analysis` recherche une couverture existante par empreinte et similarité. `visibility_analysis` compare les champs requis avec les sources de télémétrie activées. `policy_decision` applique alors une règle déterministe. Le traitement peut se terminer par les états « déjà couvert », « manque de visibilité » ou « preuves insuffisantes », ou poursuivre vers la génération.

`generate_candidate` demande une règle Sigma structurée. `validate_candidate` applique les schémas Pydantic, les contrôles Sigma, la compilation pySigma, l'analyse de doublons, les watchers et les moteurs de score. Si le résultat est satisfaisant, `queue_review` crée une proposition. Si une correction est possible, `repair_candidate` produit une nouvelle révision et retourne vers la validation. Après revue humaine, `approved` dirige une approbation vers `deployment`, une demande de modification vers la réparation et un rejet vers l'état terminal correspondant.

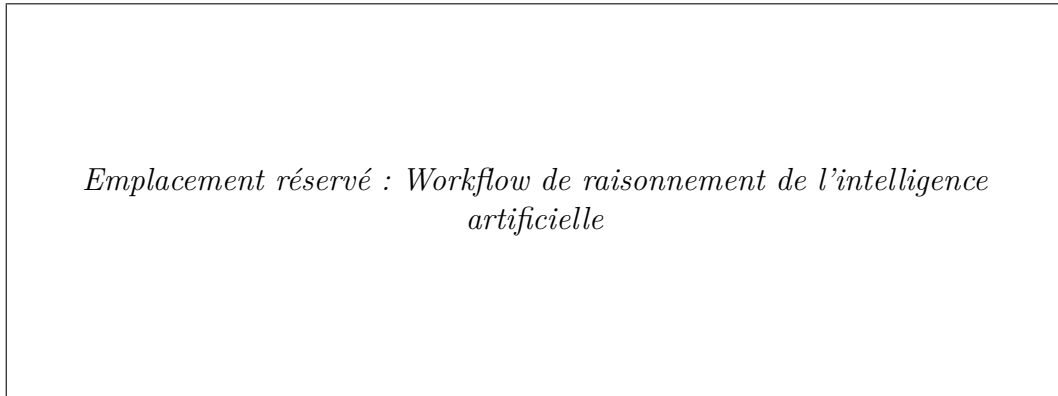


FIGURE 2.2 – Workflow de raisonnement de l'intelligence artificielle

Lorsqu'un événement contient plusieurs comportements, le nœud `advance_behavior` permet de passer au comportement suivant. Cette conception produit une proposition indépendante par comportement et évite de réunir des logiques hétérogènes dans une règle unique.

2.5.2 Révision bornée et mémoire de session

La correction n'est pas une boucle illimitée. Elle dépend de paramètres tels que le nombre maximal de révisions, le nombre maximal de réparations, le progrès minimal attendu ainsi que les cibles de confiance et de confiance déterministe. La boucle s'arrête en cas de satisfaction, d'épuisement des tentatives, d'absence de progrès significatif, de blocage de sécurité, de manque de preuve, de doublon ou d'erreur non réparable.

Chaque passage crée une entrée dans `ai_reasoning_revisions` avec la révision parente, le candidat avant et après correction, les champs modifiés, les résultats de validation, les watchers, les scores, la recommandation et la raison d'arrêt. La mémoire de session contient une synthèse structurée des comportements traités, associations acceptées ou rejetées, historiques de réparation et validation, échecs des watchers, évolutions des scores, doublons, stratégies déjà essayées, champs de télémétrie et observables utilisés.

Cette mémoire permet à une correction d'exploiter les erreurs précédentes sans enregistrer une chaîne de pensée cachée. Les invites interdisent explicitement cette dernière. Seuls les objets JSON, justifications synthétiques, résultats et candidats nécessaires à l'audit sont conservés.

2.6 Contrôles, confiance et approbation humaine

2.6.1 Watchers

Les watchers sont des contrôles indépendants produisant les états **PASS**, **WARNING** ou **FAIL**. Pour un comportement, ils vérifient le schéma, les preuves, ATT&CK, les hallucinations et la sûreté de la sortie. Pour un candidat, ils vérifient le schéma Sigma, la compilation, le niveau minimal de confiance, les doublons, la télémétrie, l'existence d'une logique de détection et l'absence d'instructions opérationnelles dangereuses. Leurs résultats sont persistés et influencent directement le routage.

2.6.2 Score de confiance

Le moteur de confiance au sens probabiliste combine les informations génératives et déterministes. Le score est défini par :

$$C = 0,25C_{IA} + 0,25V + 0,20W + 0,10T + 0,10K + 0,10D - P_r \quad (2.1)$$

où C_{IA} est la confiance annoncée par le modèle, V le score de validation, W le score des watchers, T la télémétrie, K la couverture, D l'absence de doublon et P_r la pénalité

liée aux réparations. Ce score mesure la qualité apparente du candidat, mais n'autorise jamais sa publication.

2.6.3 Score de fiabilité déterministe

Le moteur de *trust* met davantage l'accent sur les contrôles vérifiables :

$$F = 0,55V_d + 0,25C + 0,20(1 - f) - 0,03n_a \quad (2.2)$$

où V_d représente la validation déterministe, C le score précédent, f un facteur lié au nombre d'échecs et n_a le nombre d'avertissements. Une défaillance de sûreté impose un rejet et une validation échouée ne peut pas être compensée par une confiance élevée du modèle. Les recommandations possibles sont l'approbation, l'approbation avec avertissement, la revue nécessaire, la demande de changements ou le rejet.

2.6.4 Principe de contrôle humain

Une proposition satisfaisante est placée dans la file de revue. L'analyste consulte la règle, les preuves, la compilation, les scores, les avertissements et l'historique. Il peut approuver la révision courante, demander une nouvelle correction ou rejeter la proposition. Seule l'approbation reprend le graphe vers la création d'un paquet Sigma et d'une entrée dans le catalogue. Le terme « déploiement » désigne donc ici une publication locale contrôlée, et non une installation automatique dans un SIEM réel.

2.7 Déploiement conteneurisé

Docker Compose décrit PostgreSQL 16, Redis 7, le backend, le worker, le planificateur, le frontend et Nginx. Un profil ajoute MISP, MariaDB, Valkey, les modules et SMTP. Les volumes persistants conservent les données et les contrôles de santé vérifient la disponibilité des services. Alembic applique les migrations du schéma au démarrage contrôlé de l'environnement.

La séparation des réseaux est un choix important : MISP possède un réseau interne et seuls le backend, le worker et le planificateur peuvent le joindre par le nom `https://misp`. L'utilisateur accède à la plateforme par Nginx sans exposition directe des services internes.

2.8 Conclusion

L'architecture associe une interface adaptée au SOC, une API structurée, un traitement asynchrone et un graphe persistant. Les contrôles déterministes entourent les appels génératifs et l'action humaine constitue la dernière barrière. Le chapitre suivant présente la mise en œuvre, la validation et les résultats de cette conception.

Chapitre 3

Réalisation, validation et bilan

3.1 Introduction

Ce chapitre décrit la réalisation des principaux composants, l'intégration de la chaîne DevSecOps et les méthodes de test. Il présente ensuite les difficultés traitées, les résultats constatés et les limites de la version actuelle.

3.2 Mise en œuvre du backend

3.2.1 API et sécurité applicative

L'application FastAPI expose des routes versionnées pour l'authentification, les événements CTI, MISP, les exécutions LangGraph, les sessions d'intelligence artificielle, la revue et le catalogue. Les schémas Pydantic contrôlent les données en entrée et en sortie. SQLAlchemy assure la correspondance avec PostgreSQL et Alembic versionne les migrations.

L'authentification locale utilise un couple de jetons JWT d'accès et de renouvellement. Les dépendances FastAPI vérifient l'utilisateur actif et son rôle avant d'autoriser une action. Les routes de consultation et les actions sensibles, notamment l'approbation, sont ainsi séparées. Cette sécurité reste adaptée à une démonstration locale ; elle devra évoluer vers une fédération d'identité et une authentification multifacteur dans un environnement réel.

3.2.2 Ingestion MISP et idempotence

La plateforme peut lister les événements MISP, afficher un événement, ingérer une sélection ou consommer tous les nouveaux éléments. Le service de normalisation transforme

la structure reçue en événement CTI interne tout en conservant la donnée brute pour l'audit. Un index unique sur l'identifiant MISP empêche la duplication de l'événement interne.

L'idempotence est complétée par les enregistrements de consommation. Une nouvelle tentative peut être marquée `duplicate` ou `already_processing` au lieu de provoquer une insertion incohérente. Les autres états comprennent `pending`, `consumed`, `skipped`, `failed` et `invalid_source_event`. Le diagnostic distingue également l'absence de clé API, l'inaccessibilité du serveur, l'échec d'authentification et l'état sain.

3.2.3 Services déterministes

L'analyse ATT&CK confirme qu'une technique proposée existe dans le référentiel local. Le service de couverture compare l'empreinte du comportement et sa similarité aux détections du catalogue. Le service de visibilité vérifie les sources et champs requis. La politique synthétise ces résultats pour décider de poursuivre ou d'arrêter.

Pour les candidats, la validation combine le schéma, les exigences Sigma, la compilation pySigma vers Splunk, les doublons exacts et proches ainsi que les watchers. Ce découpage facilite les tests unitaires et évite d'enfouir des règles métier dans les invites du modèle.

3.3 Mise en œuvre du moteur d'intelligence artificielle

L'abstraction de fournisseur définit des contrats structurés pour l'extraction des comportements, la génération et la réparation. Le mode prédéfini renvoie des sorties reproductibles utilisées en démonstration et en intégration continue. Le mode DeepSeek construit les requêtes HTTP, applique les reprises prévues et assainit les messages d'erreur afin de ne pas exposer de données sensibles.

Chaque interaction conserve la version de l'invite, les objets de requête et de réponse, le fournisseur, le modèle, la latence, les informations de jetons et les empreintes. Ces données permettent de retrouver l'origine d'un candidat. L'utilisation exclusive de JSON structuré rend possible la validation Pydantic avant toute utilisation métier.

La réparation reçoit le candidat précédent, les erreurs, l'historique de la session et les stratégies déjà tentées. Elle doit corriger les éléments techniques sans recopier directement les commentaires libres d'un analyste dans les sélections Sigma. Après chaque correction, le candidat repasse par l'ensemble des contrôles ; aucune validation antérieure n'est supposée rester valable.

3.4 Mise en œuvre du frontend

Le frontend organise l'activité autour de vues opérationnelles. La page des menaces affiche la boîte MISP, les événements ingérés et les commandes de planification. Les vues IA présentent les sessions, révisions, watchers, scores et raisons d'arrêt. La vue du graphe restitue la chronologie des nœuds, tandis que la file de revue réunit le candidat, sa validation et les actions analyste.

Le catalogue ne contient que les règles approuvées ou importées. La matrice ATT&CK donne une vision de la couverture, l'inventaire de télémétrie permet d'activer les sources disponibles et la page de santé regroupe l'état des composants. Cette organisation rend visibles non seulement les résultats, mais aussi les éléments nécessaires à leur interprétation.

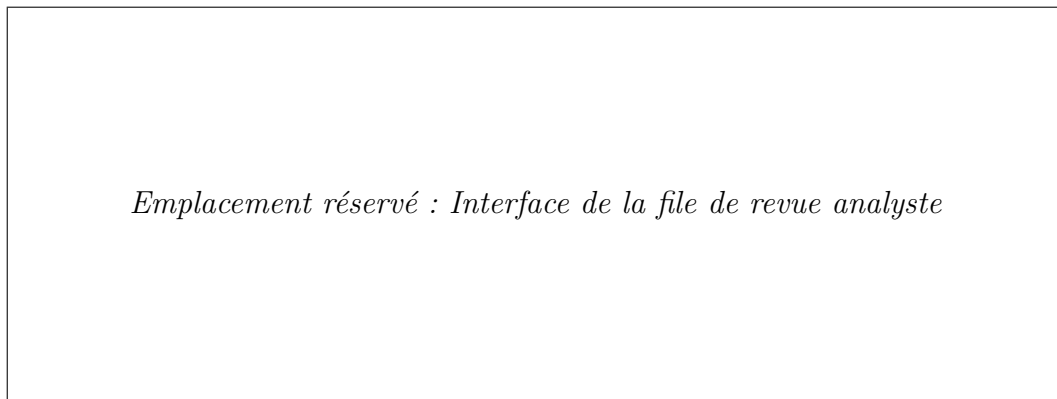


FIGURE 3.1 – Interface de la file de revue analyste

3.5 Chaîne DevSecOps et CI/CD

La démarche DevSecOps intègre les contrôles de qualité et de sécurité au cycle de développement. Cinq workflows GitHub Actions structurent cette automatisation.

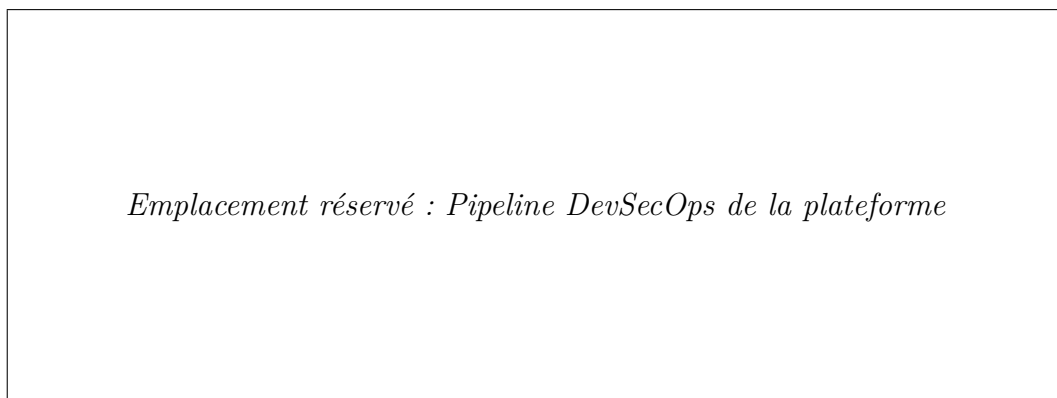


FIGURE 3.2 – Pipeline DevSecOps de la plateforme

TABLE 3.1 – Workflows d'intégration et de déploiement continus

Workflow	Contrôles principaux
Qualité des PR	Compilation Python, Ruff, formatage, mypy, tests backend, lint frontend, TypeScript et build Next.js
Validation Docker	Validation Compose, construction, démarrage PostgreSQL/Redis, vérification Alembic, migrations, santé et collecte des journaux
Intégration/E2E/DAST	Pile complète avec MISP, préparation de la clé API, scénario MISP réel, tests d'intégration, OWASP ZAP et Playwright
Sécurité	pip-audit, Bandit, detect-secrets, Semgrep, Checkov, CodeQL, Trivy, et intégrations optionnelles Snyk/SonarQube
Déploiement	Construction et publication des images backend/frontend dans GHCR, puis appel de webhooks selon l'environnement

Les contrôles couvrent plusieurs niveaux. pip-audit vérifie les dépendances Python ; Bandit, Semgrep et CodeQL recherchent des faiblesses dans le code ; detect-secrets détecte les secrets ; Checkov analyse les fichiers d'infrastructure ; Trivy inspecte les images ; OWASP ZAP exécute un contrôle dynamique de base contre la pile démarrée. Les outils Snyk et SonarQube restent conditionnés à la présence de secrets externes.

Le workflow de déploiement publie des images dans GitHub Container Registry et déclenche des webhooks. Cette automatisation ne doit pas être confondue avec l'action métier d'approbation d'une règle : dans les deux cas, des contrôles préalables sont nécessaires, mais l'un concerne l'application et l'autre le cycle de vie d'une détection.

3.6 Stratégie de test

3.6.1 Tests unitaires du backend

Les tests unitaires couvrent les contrats du mode IA prédéfini, la construction des requêtes DeepSeek, les reprises et l'assainissement des erreurs. Ils valident également les services ATT&CK, couverture, télémétrie, politique, Sigma et doublons, ainsi que les watchers, les moteurs de scores et les décisions de satisfaction.

Des tests spécifiques vérifient la compilation pySigma, la connectivité et la planification MISP, l'enregistrement des tâches Celery et l'existence d'un graphe LangGraph compilé. Un artifact JUnit conservé dans le dépôt fait état de 55 tests backend lors d'une exécution antérieure.

3.6.2 Tests d'intégration

Les tests d'intégration démarrent la pile et interrogent les API réelles. Ils couvrent la santé, le tableau de bord, MISP, la normalisation CTI, la création d'une exécution LangGraph, les métadonnées du workflow IA, les sessions, watchers, propositions et validations. Le scénario poursuit le traitement jusqu'à la revue puis vérifie la publication dans le catalogue après approbation. Les accès sans authentification sont également refusés. Les éléments du dépôt indiquent quatre tests d'intégration exécutés dans le scénario de référence.

3.6.3 Tests de bout en bout

Playwright reproduit le parcours d'un utilisateur depuis la connexion jusqu'aux principales pages : tableau de bord, menaces, intelligence artificielle, workflow, revue, catalogue, ATT&CK, santé et contrôles d'ingestion. La redirection d'un utilisateur non authentifié est vérifiée. La suite présente dans le dépôt définit trois scénarios E2E, destinés principalement à une validation de fumée de l'interface.

3.7 Difficultés rencontrées et solutions

L'intégration MISP a demandé une gestion précise de l'authentification. La préparation de la clé API en CI et la distinction entre serveur non configuré, inaccessible ou refusant l'authentification ont permis de produire des diagnostics exploitables. La duplication d'événements dans l'affichage et lors de l'ingestion a été traitée par canonicalisation des titres, contrainte d'unicité et journal de consommation idempotent.

Le démarrage conteneurisé a révélé des contraintes sur les chemins Alembic et le chargement des modules Python. L'exécution des migrations avec `PYTHONPATH=/app` et la vérification explicite des chemins dans le workflow Docker ont stabilisé cette étape.

Au niveau de l'interface, des noms de champs devenus obsolètes dans les contrats d'intégration et des sélecteurs Playwright ambigus en mode strict ont été corrigés pour aligner les tests sur les composants réellement rendus.

La difficulté principale du raisonnement concernait la continuité entre les révisions. La persistance d'une mémoire structurée et le lien parent-enfant entre tentatives ont permis de reprendre une session sans perdre les erreurs précédentes. Les erreurs pySigma, notamment l'absence de condition, sont renvoyées à la réparation. Des consignes supplémentaires empêchent la correction de recopier mécaniquement un commentaire d'analyste dans la logique Sigma.

3.8 Résultats obtenus

La plateforme réalisée couvre l'ensemble du chemin prévu : ingestion d'un événement MISP réel, normalisation, orchestration LangGraph, traitement Celery, extraction de comportements, vérification ATT&CK, analyse de couverture et de télémétrie, décision de politique, génération Sigma, compilation Splunk, contrôles, correction bornée, revue humaine et publication locale.

Sur le plan de la traçabilité, les exécutions du graphe, interactions IA, sessions, révisions, watchers, scores, décisions et actions analyste sont persistés. L'utilisateur peut donc relier une règle publiée à l'événement source, au comportement, aux contrôles et à l'approbation correspondante.

Sur le plan de la sûreté, la solution respecte la règle centrale du projet. Même lorsqu'un candidat atteint les seuils de confiance et de fiabilité, il demeure une proposition. La création d'un artifact et d'une entrée de catalogue nécessite une action d'approbation. Une défaillance déterministe ou de sécurité ne peut pas être compensée par le score annoncé par le modèle.

Les tests et workflows présents dans le dépôt apportent une preuve automatisée de plusieurs propriétés : qualité du code, migrations, disponibilité des services, contrats API, cycle de revue, sécurité statique, analyse d'images et parcours utilisateur. Ces résultats restent ceux d'une plateforme locale de démonstration et non d'une qualification pour une production à haute disponibilité.

3.9 Limites

La version actuelle comporte plusieurs limites :

- l'authentification est locale et ne fournit ni SSO ni MFA ;
- CORS autorise toutes les origines, ce qui convient uniquement à l'environnement local ;
- Docker Compose ne fournit pas la haute disponibilité ou l'orchestration d'une plateforme de production ;
- le déploiement produit un paquet et une entrée de catalogue, sans pousser la règle vers un SIEM réel ;
- le mode IA prédéfini est reproductible mais ne représente pas les variations d'un modèle connecté ;
- le mode DeepSeek dépend d'une clé et d'un accès réseau externes ;
- Snyk et SonarQube nécessitent une configuration externe ;
- OWASP ZAP exécute une analyse de base et non un test d'intrusion complet ;

- l'état des workers est approximé en partie par l'accès Redis ;
- les images MISP utilisent par défaut l'étiquette `latest` ;
- l'inventaire de télémétrie est configuré localement au lieu d'être découvert dans un SIEM ;
- la couverture E2E vérifie les parcours principaux sans constituer une régression exhaustive.

3.10 Perspectives d'amélioration

Les évolutions prioritaires concernent l'industrialisation et l'intégration. L'ajout d'un SSO, d'une authentification multifacteur, de rôles plus fins, d'une politique CORS restrictive et d'un gestionnaire de secrets renforcerait la sécurité. Le verrouillage des versions MISP améliorerait la reproductibilité.

Une observabilité plus complète pourrait réunir des journaux structurés, des identifiants de corrélation, des métriques et des tableaux de bord pour Celery, le planificateur, MISP et LangGraph. Les états des workers et du scheduler seraient alors vérifiés directement plutôt qu'estimés par Redis.

Sur le plan fonctionnel, une connexion à de véritables cibles SIEM permettrait de valider puis de déployer les règles dans un environnement contrôlé. La découverte automatique de la télémétrie, l'enrichissement du référentiel ATT&CK, l'extension des tests de contrats et des scénarios Playwright, ainsi qu'une meilleure visibilité sur les reprises IA/MISP complèteraient la solution.

3.11 Conclusion

La réalisation démontre qu'une chaîne CTI vers détection peut être automatisée sans supprimer la responsabilité de l'analyste. Les services déterministes, la mémoire de session, les révisions et les pipelines de test encadrent l'usage de l'intelligence artificielle. Les limites identifiées définissent clairement la différence entre une plateforme locale complète sur le plan fonctionnel et une solution prête pour une exploitation industrielle.

Conclusion Générale

Ce stage d'un mois réalisé au sein de Forvis Mazars Group avait pour objectif de contribuer à une plateforme d'ingénierie de détection CTI assistée par intelligence artificielle. Le besoin initial provenait du nombre d'étapes nécessaires pour transformer un événement MISP en règle exploitable : normalisation, compréhension du comportement, association MITRE ATT&CK, vérification de la couverture et de la télémétrie, génération Sigma, compilation, contrôle, revue et publication.

La solution étudiée met en œuvre une architecture complète associant FastAPI, Next.js, PostgreSQL, Redis, Celery, LangGraph, MISP et Docker. Le graphe orchestre les décisions, les services déterministes contrôlent les sorties et PostgreSQL conserve l'historique. L'intelligence artificielle intervient dans l'extraction, la génération et la réparation, mais ses réponses structurées sont systématiquement confrontées aux preuves et aux règles techniques.

L'un des principaux apports du projet est la boucle de révision bornée. Chaque tentative est persistée, évaluée par des watchers et comparée aux seuils de confiance et de fiabilité. La mémoire de session évite de répéter les mêmes erreurs, tandis que les conditions d'arrêt empêchent une correction indéfinie. Cette approche rend le raisonnement observable sans conserver de chaîne de pensée cachée.

Le second apport est la séparation explicite entre recommandation et autorisation. Une proposition ne peut rejoindre le catalogue qu'après une action humaine. Ainsi, un résultat généré n'est jamais assimilé à une décision opérationnelle. Cette règle est cohérente avec les exigences d'un SOC, où une détection doit être comprise, validée et attribuable.

Les tests unitaires, d'intégration et de bout en bout, complétés par les workflows GitHub Actions de qualité, sécurité, validation Docker et déploiement, renforcent la reproductibilité du projet. Ils ne remplacent cependant pas une qualification de production. L'authentification locale, l'absence de déploiement SIEM réel, les contrôles de santé partiels et l'orientation Docker Compose constituent notamment des limites importantes.

Ce stage m'a permis d'approfondir des compétences en architecture web, traitement asynchrone, modélisation de données, CTI, Sigma, orchestration par graphe, tests et DevSecOps. Il m'a également montré que l'intégration de l'intelligence artificielle dans un

processus de cybersécurité doit être conçue autour de contrôles explicites et non autour de la seule qualité apparente des réponses.

Les perspectives portent sur l'authentification d'entreprise, l'observabilité, la gestion des secrets, l'enrichissement de la télémétrie et d'ATT&CK, l'extension des tests et l'intégration à de véritables SIEM. Ces évolutions permettraient de passer d'une plateforme locale de démonstration à une chaîne d'ingénierie de détection plus proche des contraintes d'exploitation.

Bibliographie

- [Doc] Docker. Docker compose documentation. Documentation de conteneurisation et d'orchestration locale. Consultée dans le cadre du projet.
- [Lan] LangChain. Langgraph documentation. Documentation du moteur d'orchestration par graphe. Consultée dans le cadre du projet.
- [MIS] MISP Project. Misp documentation. Documentation technique du projet MISP. Consultée dans le cadre du projet.
- [MIT] MITRE. Mitre att&ck knowledge base. Référentiel de tactiques et techniques adverses. Consulté dans le cadre du projet.
- [OWA] OWASP Foundation. Owasp zap documentation. Documentation de l'outil de test dynamique ZAP. Consultée dans le cadre du projet.
- [Ram] Sebastián Ramírez. Fastapi documentation. Documentation du framework FastAPI. Consultée dans le cadre du projet.
- [Sig] SigmaHQ. Sigma rule specification. Documentation du format Sigma et de py-Sigma. Consultée dans le cadre du projet.
- [Ver] Vercel. Next.js documentation. Documentation du framework Next.js. Consultée dans le cadre du projet.

Annexe 1

Principales technologies

Technologie	Version	Rôle
Python	≥ 3.11	Exécution du backend et des workers
FastAPI / Uvicorn	≥ 0.111 / ≥ 0.30	API REST et serveur ASGI
Pydantic	≥ 2.7	Validation des schémas et configuration
SQLAlchemy / Alembic	$\geq 2.0.30$ / $\geq 1.13.1$	ORM et migrations
PostgreSQL	16-alpine	Source de vérité relationnelle
Redis	7-alpine	Courtier et backend de résultats Celery
Celery	≥ 5.4	Tâches asynchrones et planification
LangGraph	≥ 0.2	Orchestration du workflow
PyMISP	$\geq 2.4.190$	Accès à l'API MISP
sigma-cli / pySigma Splunk	$\geq 1.0.4$ / $\geq 2.1.0$	Validation et compilation Sigma
Next.js / React	14.2.5 / 18.3.1	Interface web
TypeScript / TailwindCSS	5.4.5 / 3.4.4	Typage et styles frontend
Nginx	1.27-alpine	Reverse proxy
MISP / MariaDB / Valkey	images officielles / 10.11 / 7.2	Source CTI et dépendances MISP

Principaux points d'API

- Authentification : POST /auth/login, POST /auth/refresh, GET /auth/me.
- MISP : consultation par GET /misp/events et GET /misp/events/{id}; ingestion par POST /misp/events/{id}/ingest et POST /misp/events/ingest-all-new.
- CTI : GET /cti-events, consultation, lancement et retraitement d'un workflow.

- Graphe et IA : consultation des exécutions, chronologies, événements, sessions, révisions, workflow et consommation.
- Revue : consultation des propositions et révisions, approbation, demande de modifications et rejet.
- Catalogue : consultation, import et téléchargement des artifacts de déploiement.
- Référentiels : télémétrie, couverture ATT&CK, décisions de politique, paramètres et santé.

Glossaire

CTI	Renseignement sur les cybermenaces décrivant des indicateurs, comportements et contextes adverses.
MISP	Plateforme de partage et de gestion d'événements CTI.
IOC	Observable technique tel qu'une adresse IP, un domaine, une empreinte ou un processus.
MITRE ATT&CK	Base de connaissances des tactiques et techniques adverses.
Sigma	Format générique de règles de détection.
pySigma	Outils Python de lecture, validation et compilation de règles Sigma.
Watcher	Contrôle déterministe appliqué à une sortie IA ou à un candidat.
Confiance	Score combinant la confiance IA et plusieurs facteurs déterministes.
Fiabilité	Score de contrôle centré sur la validation, les avertissements et la sécurité.
Proposition	Candidat de détection soumis à la revue d'un analyste.
Révision	Version immuable d'un candidat et de ses preuves de validation.
DAST / SAST	Tests dynamiques et analyses statiques de sécurité.
CI/CD	Intégration continue et livraison ou déploiement continu.